

PlüschHeld · Scroll Experience Engineering Bible – Version 1.1 (Nachtrag)

Dieser Nachtrag ergänzt das Produktionshandbuch v1.0 um Regeln, die sich im ersten vollständigen Build als **reproduzierbare Fehlerquellen** erwiesen haben. Jede Regel verweist auf den Fall im Fehlerprotokoll (`docs/LEARNINGS.md` , L1–L14) und nennt einen automatisierbaren Test.

Zu Kapitel 5 – Foto → Auge → Illustration

5.4 Die Kamera gehört in den Code, nicht ins Video. (L1, L2) Ein generiertes Transformationsvideo nie 1:1 auf Scroll mappen. Zuerst den Kamerapfad messen: Frames paarweise registrieren (ORB + ECC, **affin**, Auswahl per NCC), zu Weltkoordinaten verketteten, Zielaue tracken. Scroll steuert dann das **Log-Zoomlevel** (wahrgenommen gleichmäßig) mit eigener Easing-Kurve; der Frame-Index wird daraus invertiert, Zwischenzustände werden durch Warp des nächsten geladenen Frames erzeugt. *Test:* Augen-Overlay (`?debug-eye`) – Fadenkreuz liegt in jedem Screenshot in der Pupille.

5.5 Endframe vor Artefakten schneiden. Generierte Videos blenden am Ende oft Rahmen/Papierränder ein (hier ab Frame 218). Übergang ins nächste Kapitel mit dem letzten sauberen Frame.

5.6 Übergaben zwischen Canvas und DOM pixelgenau planen. Canvas und DOM-Ziel (z. B. Buchcover) nutzen **dieselbe Geometrie-Funktion**; der Canvas zeichnet exakt in das Zielrechteck mit derselben Zuschchnittregel wie `object-fit: cover`. *Test:* Screenshot-Differenz direkt vor/nach der Übergabe (nur Resampling-Kanten, kein Versatz).

Zu Kapitel 6 – Motion-Engineering

6.2 Einfachste Engine zuerst. Sticky + ein passiver Scroll-Listener + ein rAF-Loop + reine Funktion `sampleStory(p)` erfüllt Pinning, Scrub und Reversibilität ohne Pin-Spacer. GSAP/Lenis nur bei nachgewiesenem Mehrwert.

6.3 Asynchrones markiert „dirty“, gezeichnet wird nur im rAF. (L4) Decode-/Load-Callbacks dürfen nie direkt zeichnen; nur neu zeichnen, wenn das Ergebnis besser ist als das Gezeigte.

6.4 Jede Bühnenebene leitet ihre Sichtbarkeit aus dem Timeline-Zustand ab. (L9) Nie „einmal ausblenden, wenn Ereignis X eintritt“. Pflichttest: **Sprung an jede Anker-/Sektionsposition, bevor die Assets geladen sind** (Netz künstlich verzögern).

6.5 Layout-Kopplungen brauchen einen Reduced-Motion-Wert. (L10) Negative Margins/Überlappungen, die nur mit gepinnter Bühne Sinn ergeben, per Media-Query neutralisieren.

Zu Kapitel 8 – Digitales Buch

8.4 Buttons, Swipe, Tasten scrollen – sie setzen keinen eigenen Zustand. Scroll bleibt die einzige Quelle der Wahrheit; dadurch sind Vor/Zurück und Rückwärtsscrollen automatisch konsistent.

8.5 Perspektive relativ zur Seitenbreite ($\approx 6 \times$ Seitenbreite): angehobene Seiten wachsen sonst um 30 %+ und überdecken Headline/Steuerung.

8.6 z-Staffelung explizit: rechte Blätter `translateZ((n-i)·0.9px)`, geblätterte links `translateZ(0.6 + i·0.9px)`, Wechsel bei 90° (unsichtbar). Keine `opacity/overflow/filter` auf `preserve-3d`-Elementen.

Zu Kapitel 13/14 – Assets & Performance-Budget

13.3 Mehrere Zuschnitte statt einer Sequenz: Portrait 9:16, 3:4, Landscape 1,43:1 entlang des Look-at-Pfads geschnitten → nur ~33 kB/Frame; Poster = Frame 1 desselben Sets (Cache-Treffer, pixelgleicher Start).

14.2 Nichts, was erst beim Scrollen gebraucht wird, lädt vor dem LCP. (L5) Sequenzen erst nach `load` + `requestIdleCallback` oder beim ersten Scroll/Reload mitten in der Seite. Große JSON-Manifeste per `fetch`.

14.3 Canvas-Auflösung an der Quelle deckeln, nicht an `devicePixelRatio`. (L3) Glättungsqualität per A/B-Screenshot entscheiden („high“ kostete 3× ohne sichtbaren Gewinn).

14.4 Dekodieren begrenzen: `ImageBitmap` -LRU um die Kamera (Mobile 18, Desktop 32), `close()` beim Verdrängen.

14.5 Lighthouse zweimal messen: simulierte *und* echte Drosselung (`--throttling-method=devtools`). Next.js-Seiten werden simuliert systematisch pessimistisch bewertet (hier LCP 3,8 s simuliert vs. 0,9 s real).

Zu Kapitel 12/15 – Typografie & Accessibility

12.4 Kein `ch` für Breiten von Webfont-Text. (L6) `ch` misst die „0“ des *aktuellen* Fonts → Umbruch ändert sich beim Font-Swap. `em` verwenden.

12.5 Fallback-Metriken messen, nicht schätzen. (L6) Für variable Fonts `size-adjust / ascent-override` am tatsächlich genutzten Gewicht und an echtem Beispielttext berechnen, pro Plattform-Fallback (Times/Liberation, Noto Serif, DejaVu). *Test:* Layout mit blockierten Webfonts == Layout mit Webfonts.

12.6 Basis-Typografie nicht mit `:where()` gegen einen Reset setzen. (L7) Berechnete Werte in der QA prüfen.

15.1 Kontrast gegen jeden Hintergrund-Token prüfen, auf dem ein Text-Token vorkommt. (L12)

15.2 Globale Komponenten ohne seitenspezifische Anker. (L13) Skip-Link-Ziel als Parameter.

15.3 JS-positionierte Overlays starten `visibility: hidden` , nicht `opacity: 0` – sonst zählen sie als Layout-Shift. (L11)

Zu Kapitel 17 – Tailwind v4

17.1 Jedes eigene CSS in `@layer components` (L8); separat importierte CSS-Dateien deklarieren zusätzlich `@layer theme, base, components, utilities;`.

Zu Kapitel 18 – QA-Testmatrix (neue Pflichtfälle)

Test	Muss geprüft werden
Sprung vor Asset-Load	Jede Anker-Position direkt nach dem Laden (Netz verzögert): keine verdeckenden Ebenen, keine Leerframes
Font-Swap	Layout mit blockierten Webfonts identisch
Übergaben Canvas ↔ DOM	Pixel-Differenz vor/nach dem Wechsel
Anker-Tracking	Debug-Overlay des Zielpunkts über den gesamten Zoom
Eigene Script-Zeit	< 2 ms/Frame beim schnellen Scrollen (CDP <code>ScriptDuration</code>)
Reduced Motion	vollständiger Inhalt ohne Pinning, keine Überlappungen
Tastatur	Skip-Link, Navigation, Buch per Tab/Enter, aria-live-Ansage

Kurzfassung v1.1 – zehn Regeln

1. Referenzvideo = Motion-Spezifikation – aber die Kamera gehört in den Code.
2. Erst messen (Registrierung, Anker, Kamerapfad), dann animieren.
3. Eine Timeline, eine reine Zustandsfunktion; jede Ebene leitet ihre Sichtbarkeit daraus ab.
4. Asynchrones markiert „dirty“ – gezeichnet wird nur im rAF.
5. Nichts vor dem LCP laden, was erst beim Scrollen gebraucht wird.
6. Canvas-Auflösung an der Quelle deckeln.
7. Keine `ch`-Breiten, gemessene Fallback-Metriken – CLS 0 beim Font-Swap.
8. Jede Scroll-Kopplung im Layout braucht einen Reduced-Motion-Wert.
9. Pflichttest: an jede Position springen, bevor Assets da sind.
10. Nach jedem Build messen (Lighthouse simuliert **und** echt), aufnehmen, vergleichen, gezielt korrigieren.